Write down algorithms

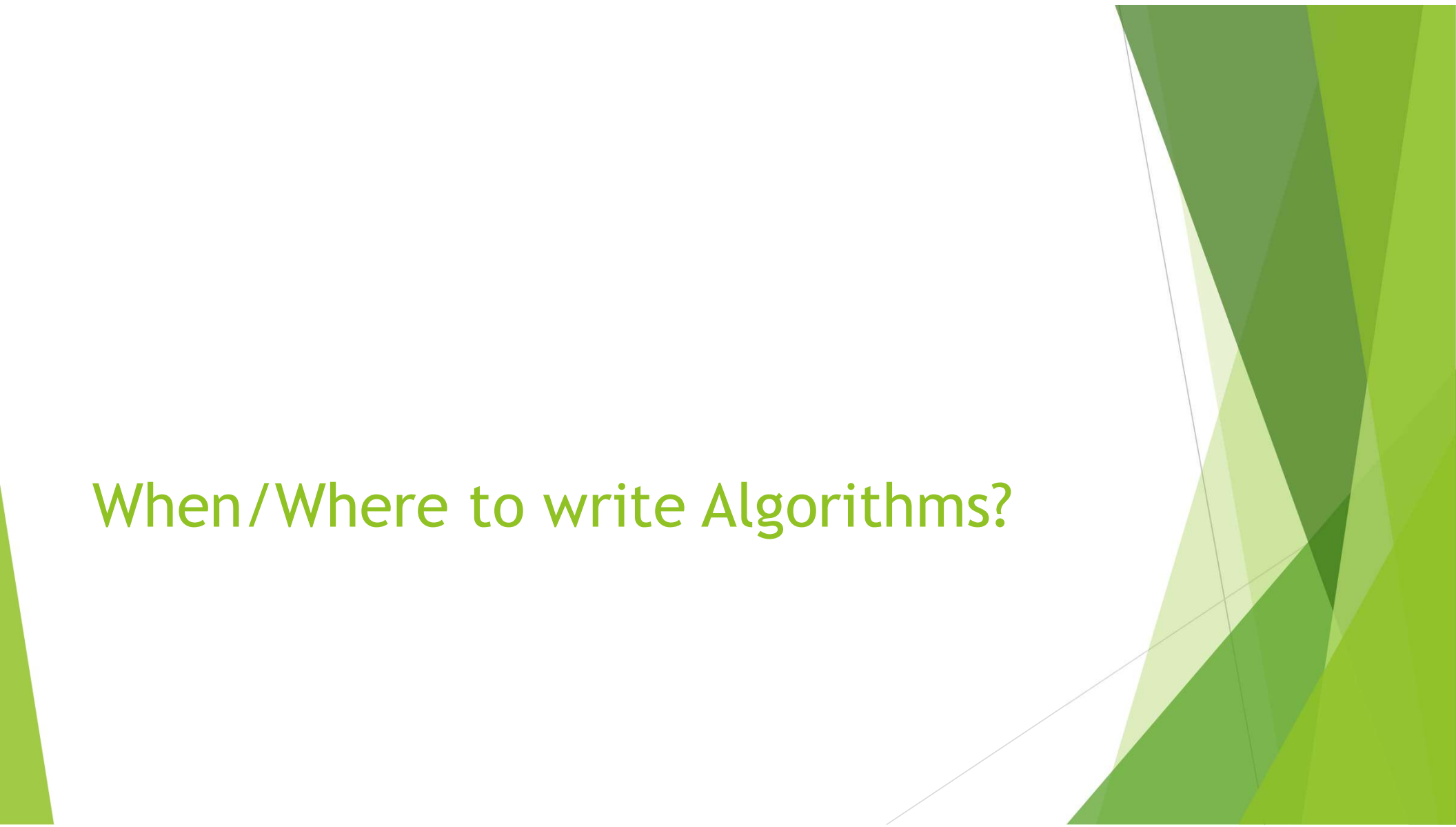
Subject: Introduction to Algorithm

Lecturer: Hoang Do Thanh Tung

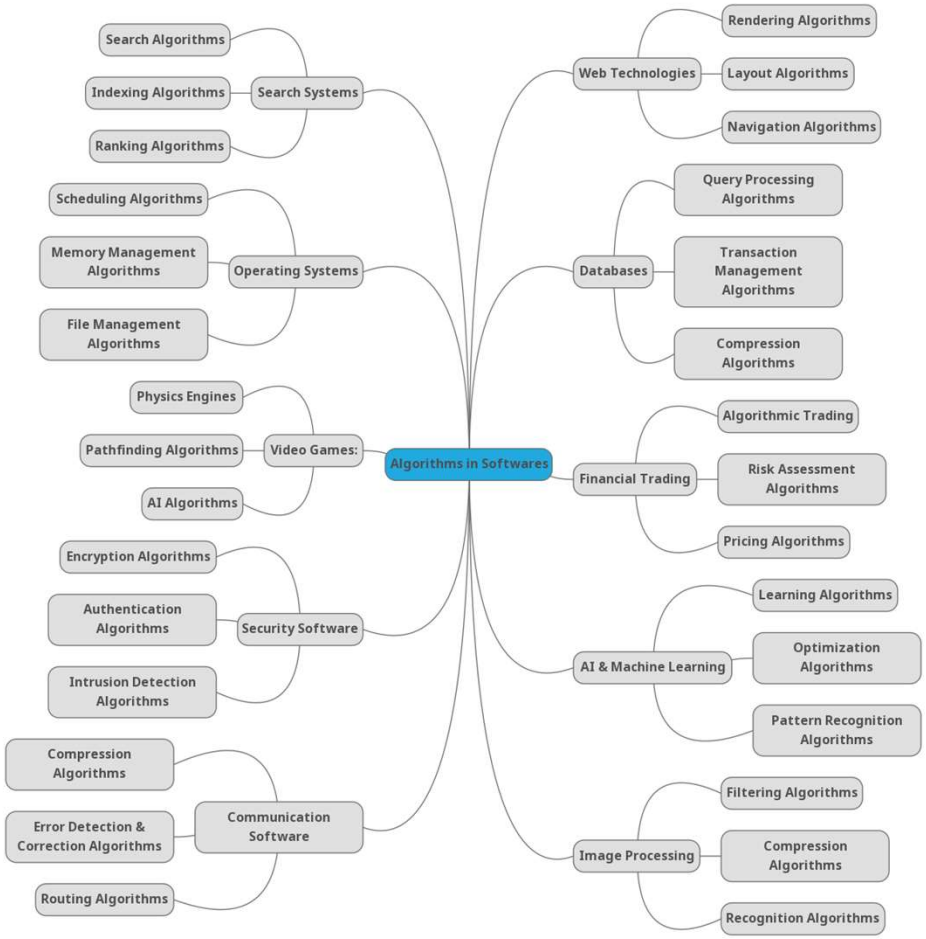
Credit points	3 ECTS
Level	Undergraduate
Teaching time Location	University of Science and Technology of Hanoi

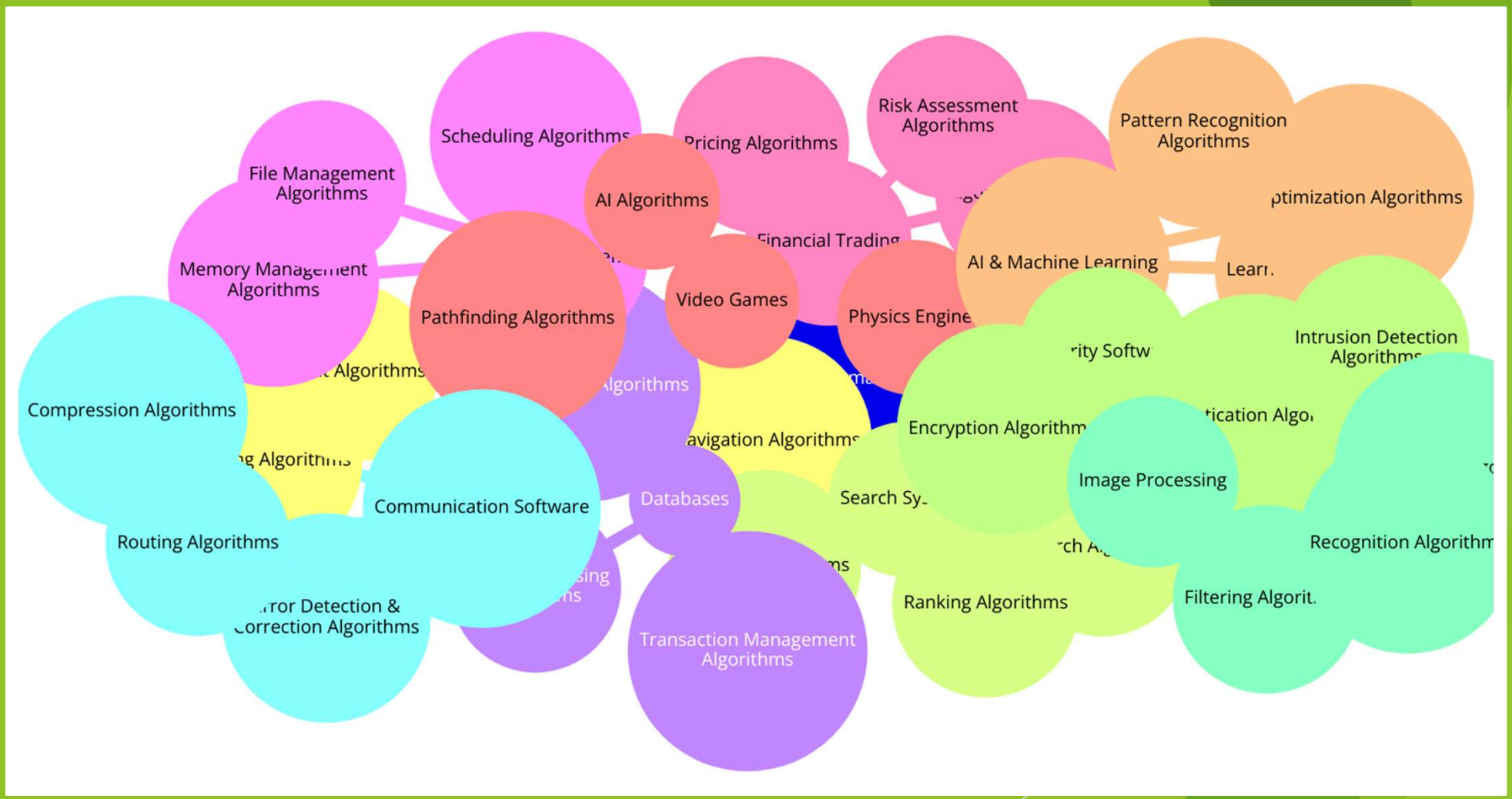
- How to write down algorithm?
- Write Pseudo-code
- Draw Flowchart
- Do Exercise

When/Where to write Algorithms?



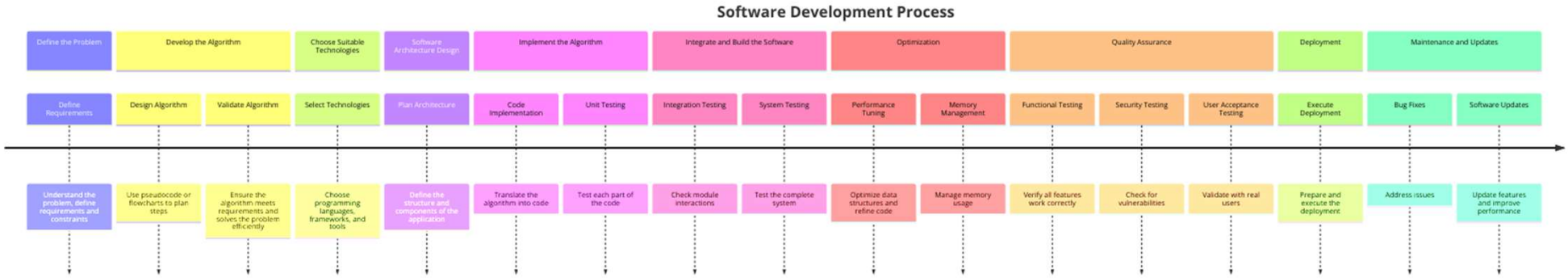
Algorithms in Software




Scheduling Algorithms
Pricing Algorithms
Risk Assessment Algorithms
Pattern Recognition Algorithms
File Management Algorithms
AI Algorithms
Financial Trading
ptimization Algorithms
Memory Management Algorithms
Pathfinding Algorithms
Video Games
Physics Engine
AI & Machine Learning
Intrusion Detection Algorithms
Compression Algorithms
Encryption Algorithm
Image Processing
Communication Software
Databases
Search Sy
Routing Algorithms
Recognition Algorithm
Ranking Algorithms
Filtering Algorit
Transaction Management Algorithms

Stage	Description	Deliverables
1. Define the Problem	Clearly understand the problem and its requirements.	- Problem statement & requirements document - User stories (optional)
2. Develop the Algorithm	Design, validate, and document the algorithm.	- Algorithm design (pseudocode or flowchart) - Validation report ensuring correctness and efficiency
3. Choose Technologies	Select programming languages, frameworks, libraries, and tools.	- Technology stack selection document - Justification based on platform, performance, and developer skills
4. Software Architecture Design	Plan the application structure and component interaction.	- Software architecture document - Diagrams illustrating component interactions
5. Implement the Algorithm	Translate the algorithm into code using the chosen language.	- Clean, well-documented code - Unit tests for each code segment
6. Integrate and Build the Software	Combine the algorithm's code with other components.	- Integrated software build - Integration test reports
7. Optimization	Analyze performance and optimize the software.	- Performance reports - Optimized code with improved efficiency and memory usage
8. Quality Assurance	Conduct thorough testing to ensure quality.	- Functional testing reports - Security testing reports - User acceptance testing reports
9. Deployment	Prepare and execute the deployment to production.	- Deployment plan with rollback procedures
10. Maintenance and Updates	Maintain and update the software after deployment.	- Bug fixes for reported issues - Updates with new features, improved performance, and security patches

How to develop a Software?


Software Development Process
Define the Problem
Develop the Algorithm
Choose Suitable Technologies
Software Architecture Design
Implement the Algorithm
Integrate and Build the Software
Optimization
Quality Assurance
Deployment
Maintenance and Updates
Define Requirements
Design Algorithm
Validate Algorithm
Select Technologies
Plan Architecture
Code Implementation
Unit Testing
Integration Testing
System Testing
Performance Tuning
Memory Management
Functional Testing
Security Testing
User Acceptance Testing
Execute Deployment
Bug Fixes
Software Updates
Understand the problem, define requirements and constraints
Use pseudocode or flowcharts to plan steps
Ensure the algorithm meets requirements and solves the problem efficiently
Choose programming languages, frameworks, and tools
Define the structure and components of the application
Translate the algorithm into code
Test each part of the code
Check module interactions
Test the complete system
Optimize data structures and refine code
Manage memory usage
Verify all features work correctly
Check for vulnerabilities
Validate with real users
Prepare and execute the deployment
Address issues
Update features and improve performance

How to develop an Algorithm?



Define the Problem
Inputs – Processing Steps – Outputs



Outline the Solution
Break problem into smaller tasks or steps
Major: Processing Steps, Subtasks, Control Structures, Variables, Record Structures, Mainline Logic

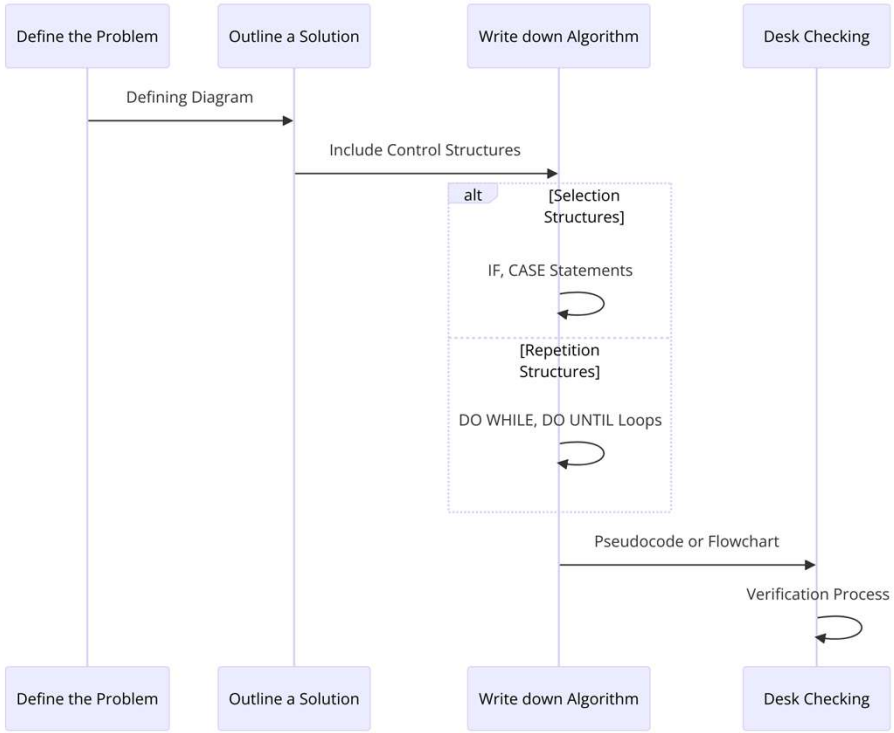


Write down Algorithms
Pseudocode, Flowcharts



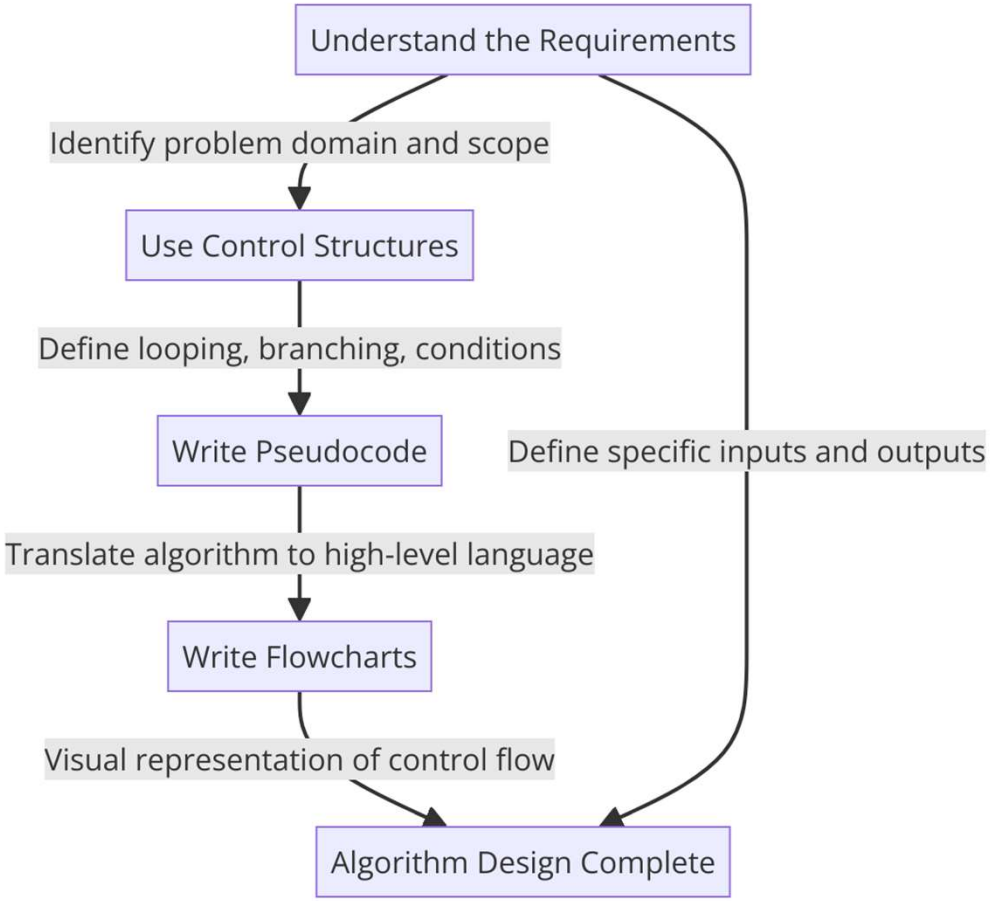
Analyze the Algorithms for correctness, Running time
Desk Checking; correct for *all possible* inputs, how fast they run

What we are studying in this



- (1) Defining the problem
  - Defining Diagram
- (2) Outline a Solution and Control Structures
  - Sequence *(sequential instructions)*
  - Selection *(IF, CASE)*
  - Repetition *(DO WHILE, DO UNTIL)*
  - Variables
  - Constants
  - Literals
- (3) Write down Algorithm
  - Pseudocode or Flowchart
- (4) Desk Checking

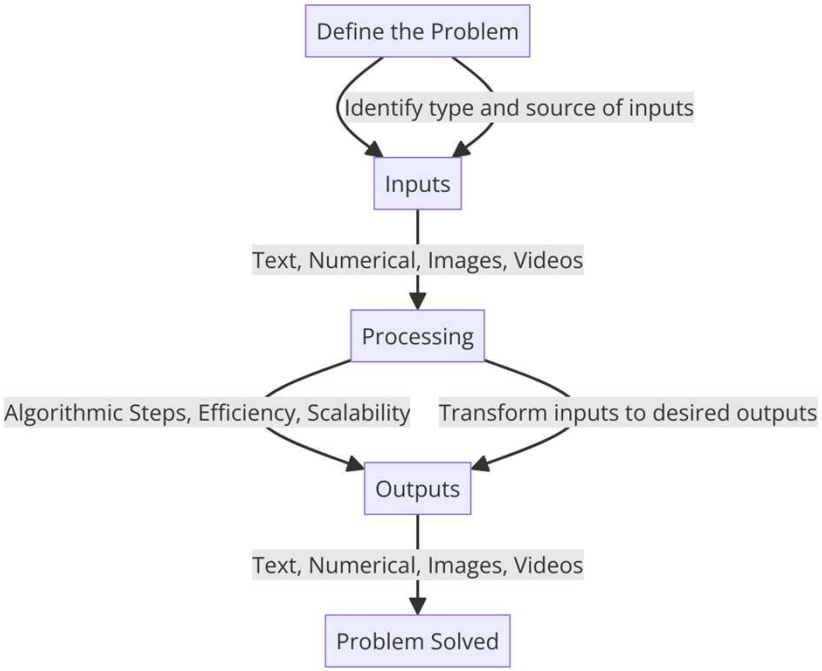
Designing an Algorithm (skills)



Define the Problem and Outline the Solution



Defining the Problem and Solution



- The problem must be defined in terms of:
  - **Input**: Data to be processed.
  - **Output**: The expected result.
    - Look for nouns in the problem statement that suggest output and input.
  - and **processing**: The statements to achieve.
    - Look for verbs to suggest processing steps.

Example 1
Calculate Area and Perimeter of a rectangle

- **Input**
  - Length
  - Width
- **Output**
  - Area
  - Perimeter
- **Processing**
  - Calculate Area = length*width
  - Calculate Perimeter = 2*(length + width)

Example 2
Calculate Sum and Average of 5 numbers

- **Input**
  - five number x1, x2, x3, x4, x5
- **Output**
  - Sum
  - Average
- **Processing**
  - Calculate Sum = x1+x2+x3+x4+x5
  - Calculate Average = Sum/5

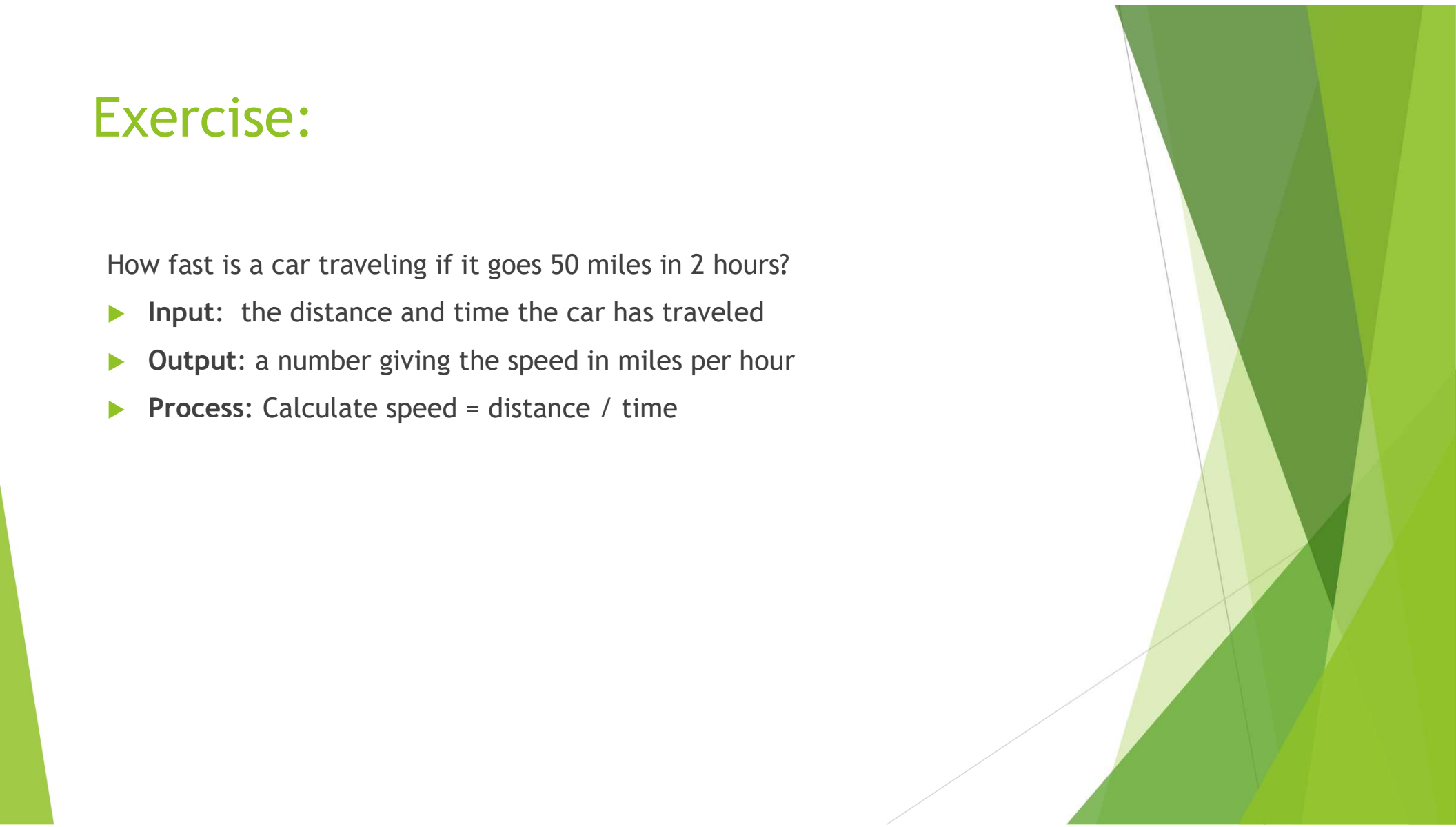
Example 3
Calculate Area and Perimeter of a circle

- **Input**
  - Radius
  - PI
- **Output**
  - Area
  - Perimeter
- **Processing**
  - Calculate Area = PI * Radius * Radius
  - Calculate Perimeter = 2 * PI * Radius

Exercise:

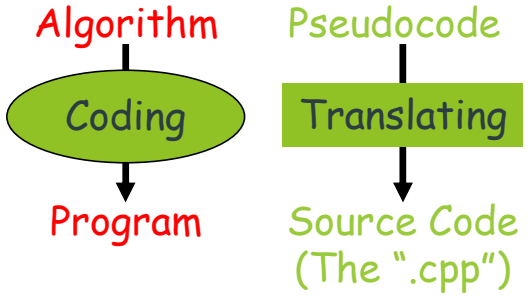
How fast is a car traveling if it goes 50 miles in 2 hours?

- **Input:** the distance and time the car has traveled
- **Output:** a number giving the speed in miles per hour
- **Process:** Calculate speed = distance / time

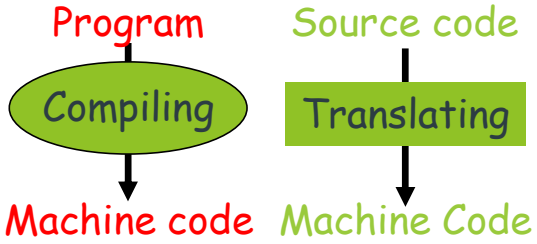


What is the rest?

- **Write down Algorithms**
  - **Pseudo code**
  - **Flowcharts**
- **Coding the Program** in a formal language

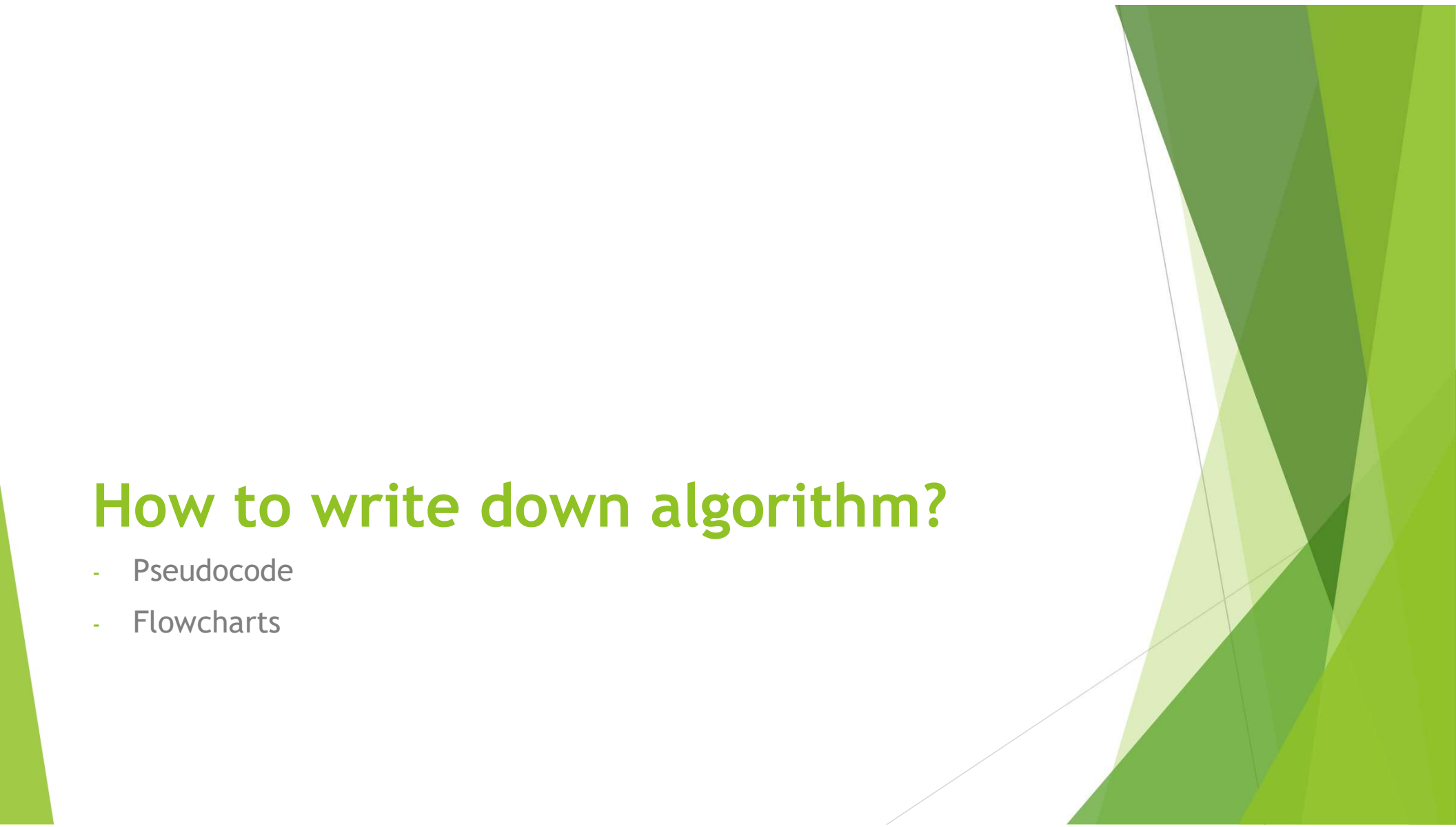


- **Compiling the program** *into machine code*



How to write down algorithm?

- Pseudocode
- Flowcharts



How to write Algorithms



Properties of Algorithms

Computer programs are concrete representations of algorithms , but algorithms are not program

Algorithms can be used in any programming language

Algorithms have a lot of structure, ex: conditionals, loops, and recursion…



How to write down in detail an algorithm with?

Natural language

Programming language

Pseudocode

Flowcharts

How to write Algorithms



Natural language

Full of ambiguities, subtleties, and shades of meaning, but algorithms must be described as accurately as possible



Programming language

C, C++, C#, Java, Python, Ruby, Erlang, Haskell, OcaML, Scheme, Visual Basic, Smalltalk, Javascript, Processing, Squeak, Forth, TEX, Fortran, COBOL, Intercal, or Brainfuck …

We should only focus on algorithm design

Write down with Pseudo-code



What is Pseudocode?

- Definition: Pseudocode is a method used for describing algorithms that mixes elements of programming syntax and natural language.
- Purpose: It aids in designing algorithms that are easy to understand and implement without the constraints of specific programming syntax.

Key Components of Pseudocode



Programming Structures: Utilizes loops, conditionals, functions, and variables to outline logical structure.



Mathematical Notation: Employs mathematical symbols for precise operation description.



Natural Language: Incorporates plain language descriptions to enhance accessibility and comprehension.

Advantages of Pseudocode in Algorithm Design



Clarity and Simplicity: Focuses on essential steps without complex syntax.



Hides Irrelevant Details: Concentrates on algorithm logic rather than implementation specifics, aiding in planning and problem-solving.



Ease of Analysis and Debugging: Simplifies logical error analysis and conceptual debugging.



Implementation Flexibility: Facilitates implementation in any programming language, enhancing adaptability across various platforms.

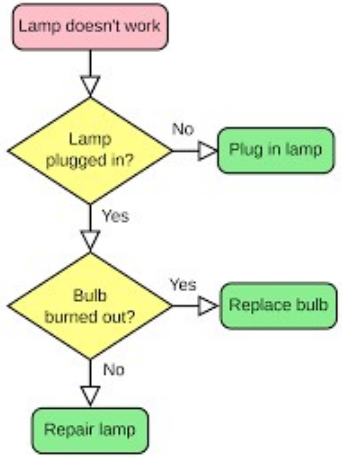
Pseudocode styles

- There are many styles of pseudocode
  - **Narrative-style:** Similar to storytelling, it describes the algorithm's steps in plain English, making it easy to read for anyone.
  - **Structured-style:** Borrows heavily from programming languages, using keywords like "if," "for," and "while" to structure the logic clearly.
  - **Flowchart-style:** Uses flowcharts with boxes and arrows to visually represent the algorithm's flow. Great for people who prefer a visual representation.
  - **Outline-style:** Breaks down the algorithm into a hierarchical list, with subtasks nested under main steps. Good for organizing complex algorithms.
  - **Mathematical-style:** Leverages mathematical notation alongside natural language to describe steps with complex calculations. Useful for algorithms heavily reliant on math.

1. Narrative-style:

Algorithm: Calculate the average of a list of numbers.

•Start with an empty variable to store the sum.
•Loop through each number in the list.
• Add the current number to the sum.
•Divide the sum by the total number of elements in the list.
•Store the result as the average.
•Print the average.



2. Structured-style:

Function average(numbers):

•total_sum = 0
•FOR each number in numbers:
•total_sum = total_sum + number
•average = total_sum / length(numbers)
•RETURN average

4. Outline-style:

Algorithm: Calculate the average of a list of numbers.

1.Initialize a variable total_sum to 0. 2. Iterate through each number in the list.
1.1 Add the current number to total_sum.
2.Calculate the average by dividing total_sum by the number of elements in the list.
3.Display the calculated average.

5. Mathematical-style:

Algorithm: Calculate the average of a list of numbers (n elements).

•Define average = Σ(x_i) / n, where x_i represents each number in the list.

Tips and tricks of pseudocode

- Start with the problem statement: clearly stating the problem and the inputs and outputs of the algorithm.
- Keep it simple: Use simple language and avoid unnecessary complexity.
- Use a consistent style: Use a consistent style throughout your pseudocode
- Break down the problem into smaller steps: Divide the problem into smaller, more manageable tasks.
- Use clear and descriptive variable names: Use variable names that accurately describe the data they represent.
- Include comments: Add comments to your pseudocode to explain complex or non-obvious steps.

A good Pseudocode

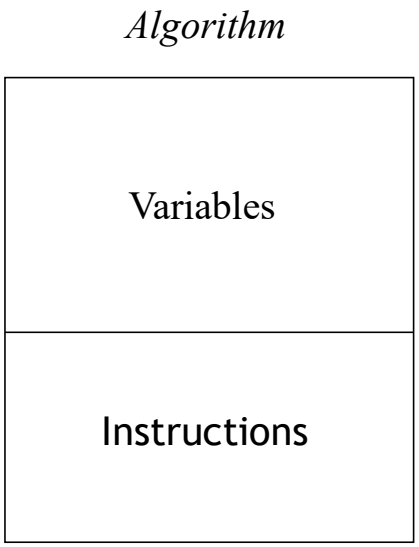
```
// Algorithm to find the largest number in an array
// Input: An array of integers
// Output: The largest integer in the array

function findLargestNumber(array)
    // Initialize the largest number with the first element
    largestNumber = array[0]

    // Go through each element in the array
    for each element in array
        // If the current element is greater than the largest number
        if element > largestNumber
            // Update the largest number
            largestNumber = element

    // Return the largest number
    return largestNumber
```

Structure of an algorithm



1.Input: receiving input data-numbers and text to more complex data structures like lists or graphs.
2.Initialization: setting up variables, data structures, or any other elements needed for the subsequent processing.
3.Processing: The processing steps manipulate the input data to achieve the desired outcome-calculations, comparisons, conditional branching, loops, and function calls.
4.Output: produces an output.
5.Termination: Reaches a termination point. This typically involves exiting the loop or function that defines the algorithm.

- An algorithm consists of instructions
- Instructions involve variables

Example: Finding the Largest Number in an Array

Input: An array of integers arr = [3, 1, 4, 1, 5, 9]

Initialization: maxNum = arr[0]

Processing:

```
for i from 1 to length(arr) - 1
    if arr[i] > maxNum
        maxNum = arr[i]
```

Output: maxNum

Termination: Return maxNum

Characteristics of Algorithms

- Finiteness: Must have a finite number of steps.
- Definiteness: Each step must be clear and unambiguous.
- Effectiveness: Must effectively solve the problem.
- Efficiency: Should be efficient in terms of time and space complexity.

Instructions of an algorithm

- **Basic (primitive) instructions**
  - Read the input from user
  - Print the output to the user
  - Cary out basic arithmetical computations
- **Conditional instructions**
  - Execute an instruction (or a block of instructions) if a condition is true
- **Repeat instructions**
  - Execute a block of instructions (multiple) times until a certain condition is met
- and…**variables**

Variables and Arrays…

- Computers work with data (numbers, words, etc)
- Data must be stored (in variables)
- Each variable is assigned a storage “box”
  - can store one number at any time
  - eg: sum, j, carry
- Arrays:
  - Often deal with many numbers
  - Such as A_1, A_2, A_3, … , A_{100}
  - Store as an “array” A[1], A[2], … , A[100]
    - we treat each of them as a variable,
    - each is assigned a storage “box”

Basic instructions (1)

Assign values to variables using basic arithmetic operations

Example: Let's assume we have two variable, a and b.

- We execute:
  - `Set the value of a  to 3`

  This instruction changes the value of a to 3.
- Now we execute
  - `Set the value of b to a*3 + 12`

  This instruction changes the value of b to 3+3 + 12 = 21

Basic instructions (2)

Read input from user

Example:

```
get the value of [variable] a [from user]
```

- When this instruction is executed,
  - the algorithm stops,
  - waits for the user to type in an answer, and
  - the value typed by the user is stored into variable a.

Basic instructions (3)

3. Print to user (screen)

Print variables and/or text

Example:

```
print the value of variable x
```

- When a print instruction is executed, the algorithm writes to screen the value of variable x

```
print "Hello world!"
```

- This instruction prints to screen the text "Hello world!"

- Why do we need quotation signs?
  - `Print hello`
  - `Print "hello"`

Basic instructions (4)

- **Incorrect**
  - `Set 1 to a`
  - `Add a + b` (what do you do with the result?)
  - `Set a+b+3`
- **NOT the same**
  - `set a to b`
  - `set b to a`
- `Example:` what is the output of the following algorithms?

```
set a to 2          set a to 2
set b to 4          set b to 4
set a to b          set b to a
print a, b          print a, b
```

Example

Write an algorithm that asks the user for three numbers, computes their sum and average and outputs them.

One possible algorithm for this problem:

```
Variables: a,b,c, sum, avg
```

- `Get the values of a, b, c from user`
- `Set avg to (a+b+c)/3`
- `Set sum to (a+b+c)`
- `Print sum, avg`

Example

Write an algorithm that reads the value of a circle radius from the user, and prints the circumference of a circle with that radius.

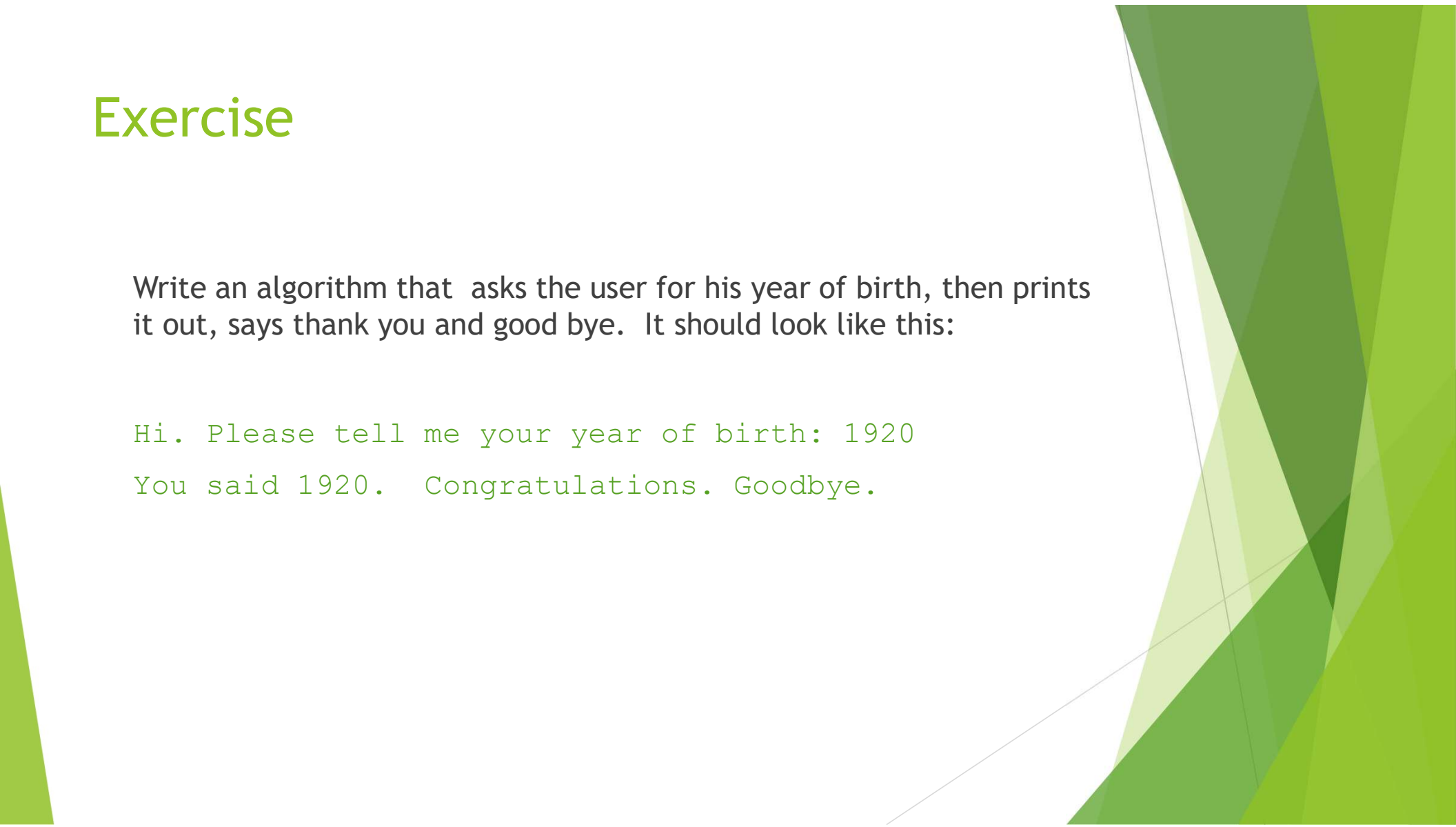
One possible algorithm for this problem:

```
variables: r, c
1. Get the value of r from user
2. Set c to 2 * pi * r
3. Print "The circumference of your circle is "  c
```

Exercise

Write an algorithm that asks the user for his year of birth, then prints it out, says thank you and good bye. It should look like this:

```
Hi. Please tell me your year of birth: 1920
You said 1920.  Congratulations. Goodbye.
```



Conditional Instructions

An instruction depending whether the condition is true or false. The else part is optional.

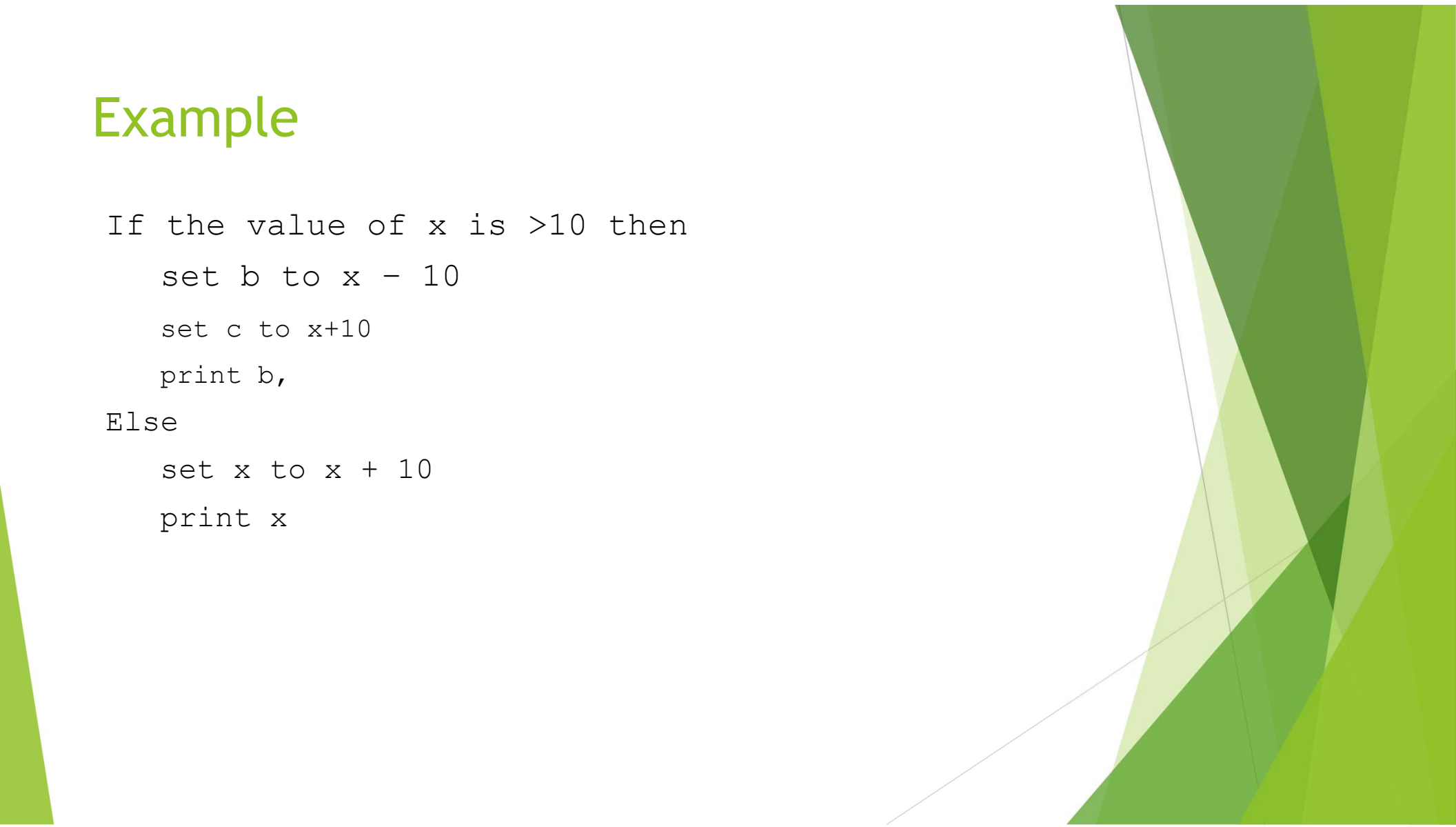
```
if <condition> then
   <instruction(s) to be done>
else
   <instruction(s) to be done otherwise>
```

Example

```
if the value of x is 0 then
    set the value of a to 0,
else
    set the vale of a to a+1
```

Example

```
If the value of x is >10 then
    set b to x – 10
    set c to x+10
    print b,
Else
    set x to x + 10
    print x
```



Example

Consider the following algorithm:

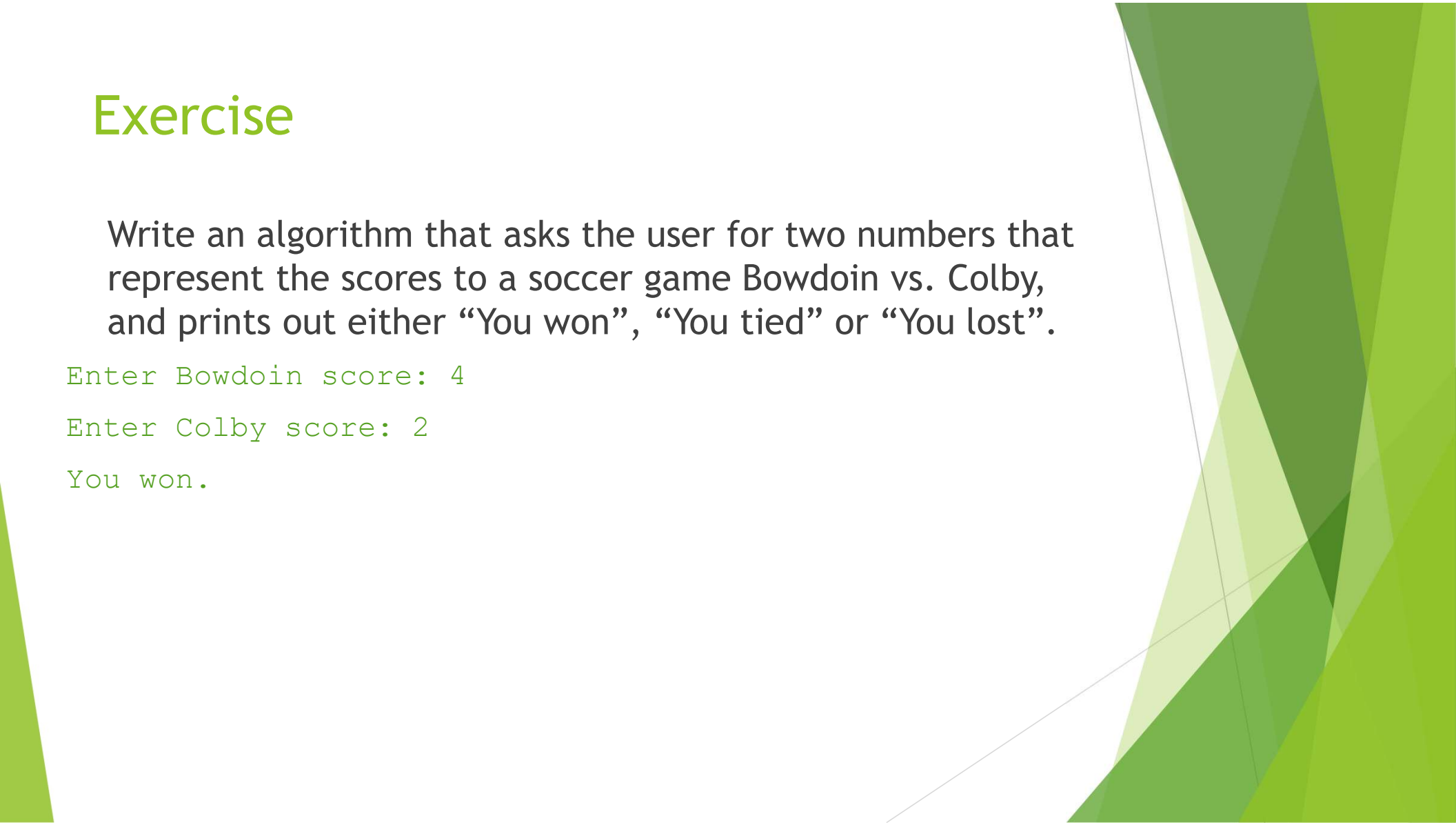
```
print "Enter two values"
get a, get b
if the value of a is larger the value of b then
   print a
else
   print b
```

What does the program print if the user enter 10, 13?

Exercise

Write an algorithm that asks the user for two numbers that represent the scores to a soccer game Bowdoin vs. Colby, and prints out either “You won”, “You tied” or “You lost”.

```
Enter Bowdoin score: 4
Enter Colby score: 2
You won.
```



Compact Pseudocode

- This is correct

```
Get the value of a from user
Get the value of b from user
If the value of a is equal to the value of b then
    print the value of a
else
    set the value of c to the value of a + the value of b and
print c
```

- But, we'll slowly converge towards a more compact, math-like style

```
Get a, get b
If (a==b) then
    print a
Else
   set c = a + b
   print c
```

```
ALGORITHM
1. Read (S, E, G);
2. Distance ← (E-S);
3. Average ← D/G;
4. Print Average;
5. Stop
```

Compact Pseudocode

- Examples:
  - `If (a >= b) …`
  - `If (a + b <= c)…`
  - `If (x == y)…`
  - `If (the remainder of x divided by y == 0) …`
    - Note: the operator for “remainder” is denoted %
  - `If (x%2 == 0) print “even number”`

Example

Write an algorithm to determine whether or not a driver is good if distance/gallons is greater 25 on a trip. An odometer will show number of gallons used and the starting and ending mileage.

```
Variables: gallons, start, end, distance, mpg
get gallons, start, end
distance = end - start
mpg = distance / gallons
print mpg
if mpg > 25.0 then
    print "You are getting good gas mileage"
else
    print "You are NOT getting good gas mileage"
```

Example

Equivalent:

- `Set the value of a to 1`
- `Set a to 1`
- **`a=1`**

Equivalent

- `Add 1 to count`
- `Set count to count + 1`
- `Increment the value of count by 1`
- **`count = count + 1`**

Writing in pseudocode gives you the freedom to choose any of these. In general people prefer the shorter form.

Combining Conditions

- Use logical operators: `and, or, not`
  - `If (condition 1 and condition 2)`
    - True if BOTH are true
  - `If (condition 1 or condition 2)`
    - True if at least one is true
  - `If (not condition 1)`
    - True if condition 1 is not true

- Examples
  - `If (score > 85 and score < 95) then print "grade is A-"`
  - `If (not (score < 2)) print "you passed"`

Exercise

Write an algorithm that asks the user for an exam score, which is expected to be a number between 0 and 100. If the score does not fit in this range, display an error message. Otherwise assign a letter grade A, B, C, D or F. The breaks are 90, 80, 70, 60. For example:

```
Please enter a score, between 0 and 100: 110.
Sorry, that's impossible.

Please enter a score between 0 and 100: 85
The grade is B.
```

Looping Primitive – while-loop

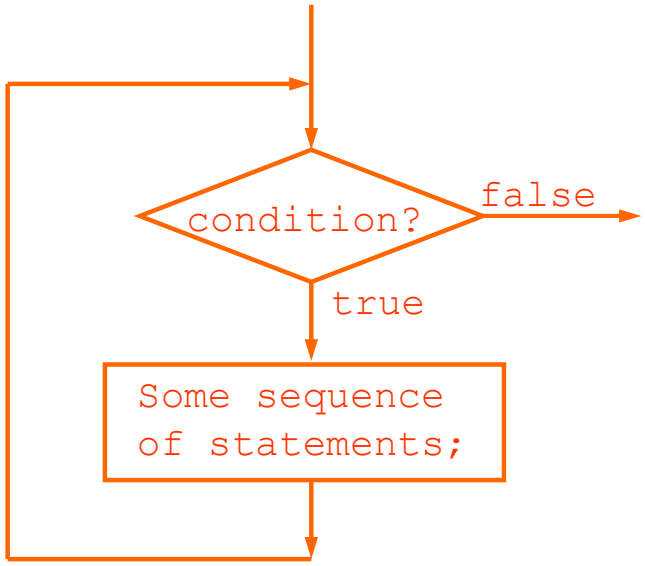
- The while-loop
  - loop a “variable” number of times
- Syntax

  while (condition) do

  (some sequence

  of statements)

  endwhile



Exercising a while loop

```
j ← 1;
while (j <= 3) do
  print j;
  j ← j + 1;
endwhile
print "--- Done ---"
```

```
(* General Loop *)
Read(n);
j ← 1;
while (j <= n) do
  print j, A[j];
  j ← j + 1;
endwhile
print "--- Done ---"
```

```
Output:
  1
  2
  3
--- Done ---
```

Looping Primitive – for-loop

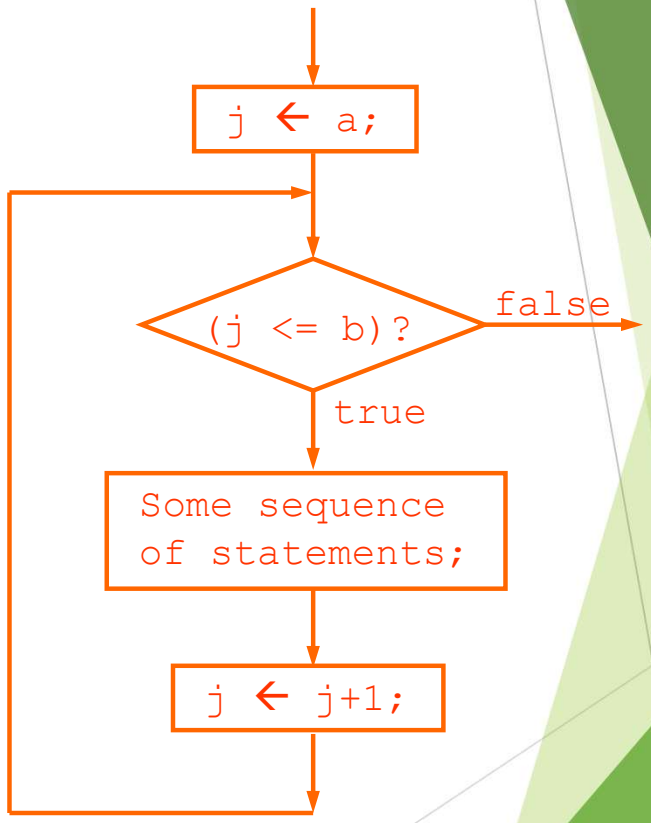
- First, the for-loop
  - loop a "fixed" or (pre-determined) number of times
- Syntax

for j ← a to b do

(some sequence

of statements)

endfor



Exercise

```
for j ← 1 to 3 do
  print j;
endfor
print "--- Done ---"
```

```
Output:
  1
  2
  3
--- Done ---
```

Exercise

```
for j ← 1 to 4 do
  print 2*j;
endfor
print "--- Done ---"
```

```
Output:
  2
  4
  6
  8
--- Done ---
```

```
j ← 1;
while (j <= 4) do
  print 2*j;
  j ← j + 1;
endwhile
print "--- Done ---"
```

```
Output:
  2
  4
  6
  8
--- Done ---
```

Simple iterative algorithm: Sum

- Given: List of numbers: A_1, A_2, A_3, …., A_n
- Output: To compute the sum of the numbers

Note: Store numbers in array A[1], A[2], … , A[n]

```
Sum(A, n);
begin
  Sum_sf ← 0;
  k ← 1;
  while (k <= n) do
    Sum_sf ← Sum_sf + A[k];
    k ← k + 1;
  endwhile
  Sum ← Sum_sf;
  Print "Sum is", Sum
end;
```

Exercising Algorithm Sum:

Input:

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	n=6
2	5	10	3	12	24	

Processing:

k	Sum-sf	Sum
?	0	?
1	2	?
2	7	?
3	17	?
4	20	?
5	32	?
6	56	?
7	56	56

Output:

```
Sum is 56
```

Algorithm for Sum (with for-loop)

- We can also use a while-loop instead of a for loop.

```
1. Sum(A, n);
2. (* Find the sum of A1, A2,…, An. *)
3. begin
4.    Sum_sf ← 0;
5.    for k ← 1 to n do
6.       Sum_sf ← Sum_sf + A[k];
7.    endfor
8.    Sum ← Sum_sf;
9.    Print "Sum is", Sum
10.end;
```

- **Exercise:** **(a) Note the differences…**
  (b) Modify it to compute the average?

Remarks about the iterative algorithm…

- Note the three stages:
  1. Initialization
     - Set some values at the beginning
  2. Iteration
     - This is the KEY STEP
     - Where most of work is done
  3. Post-Processing or Cleanup

- Can use this setup for other problems
  - Calculating average, sum-of-squares
  - Finding max, min; Searching for a number,

Another Example of Algorithm (with loops)

PROBLEM: Start with a collection of names N_1, N_2, ..., N_{10000}, and corresponding telephone numbers T_1, T_2, ..., T_{10000}. Given a name, Name, find a telephone number for that name if a match on an Ni occurs; otherwise, print "Not Found".

Note: In the book, subscripts are used for N_1, N_2, etc.

A FIRST ATTEMPT AT A SOLUTION TO THE TELEPHONE SEARCH PROBLEM

1. Get values for N_1, N_2, ..., N_{10000}, T_1, T_2, ,,,, T_{10000}, and Name.
2. if Name is N_1, then print T_1 ; Stop endif;
3. if Name is N_2, then print T_2; Stop; endif;
4. If Name is N_3 then print T_3; Stop; endif;

. . . .

{a lot of tedious writing here that is being skipped}

. . . .

10001. If Name is N_{10000}, then print T_{10000} ; Stop; endif

10002. Print "Not found"

10003. Stop.

A SECOND ATTEMPT AT A SOLUTION TO THE TELEPHONE SEARCH PROBLEM

1. Get values for N_1, N_2, ..., N10000, T1, T2, ,,,, T10000, and Name.
2. Set the value of i to 1 and the value of Found to NO.
3. Repeat steps 4 through 7 until (Found is Yes)
4. If Name is equal to Ni, then
5. Print the telephone number Ti
6. Set the value of Found to Yes

   Else
7. Add 1 to the value of i
8. Endif
9. Stop.

ANOTHER ATTEMPT AT A SOLUTION TO THE TELEPHONE SEARCH PROBLEM

1. Get values for N1, N2, ..., N10000, T1, T2, ,,,, T10000, and Name.
2. Set the value of i to 1 and the value of Found to NO.
3. Repeat steps 4 through 7 until (Found is Yes) or (i > 10000)
4. If Name is equal to Ni, then
5. Print the telephone number Ti
6. Set the value of Found to Yes

   Else
7. Add 1 to the value of i
8. If (Found is No) then
9. Print "Not found"
10. Stop.

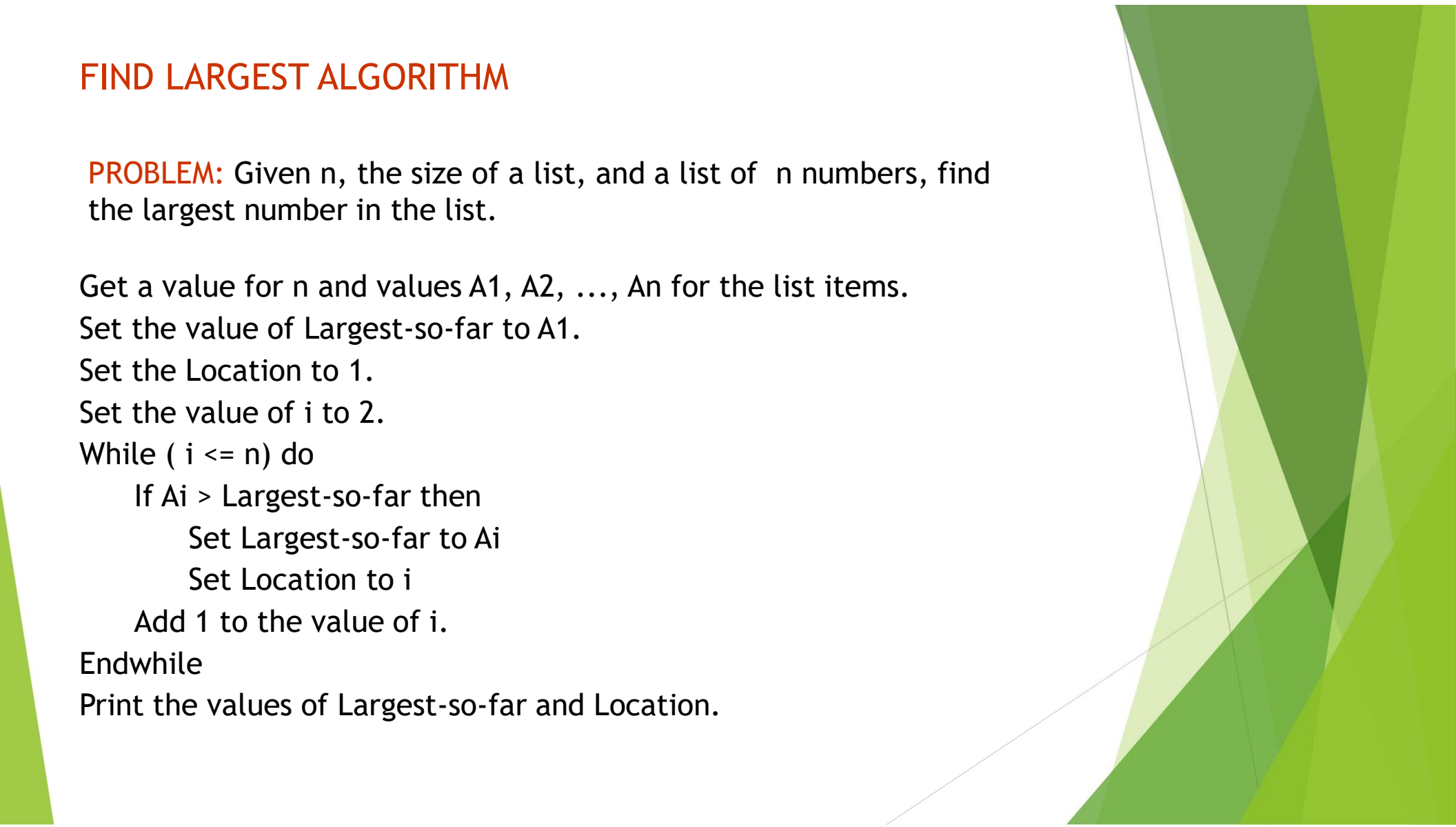
Solution to Telephone Search Problem (Using a while loop)

Get values for N_1, N_2, ..., N_{10000}, T_1, T_2, ,..., T_{10000}, and Name.
Set the value of i to 1;
Set the value of Found to “NO”;
While (Found = “No”) and (i <= 10000) do
    If (Name = N_i) then
        Print the telephone number T_i ;
        Set the value of Found to “Yes”;
    Else Add 1 to the value of i;
Endwhile
If (Found = “No”) then
    Print "Not found";

FIND LARGEST ALGORITHM

PROBLEM: Given n, the size of a list, and a list of n numbers, find the largest number in the list.

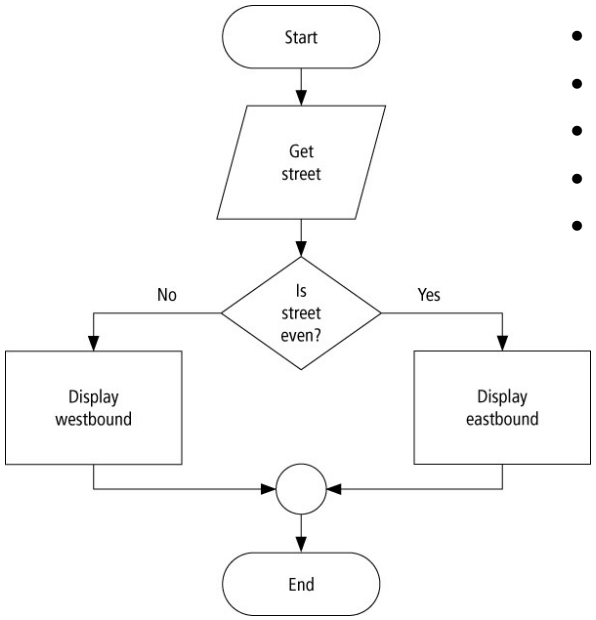
Get a value for n and values A1, A2, ..., An for the list items.
Set the value of Largest-so-far to A1.
Set the Location to 1.
Set the value of i to 2.
While (i <= n) do
    If Ai > Largest-so-far then
        Set Largest-so-far to Ai
        Set Location to i
    Add 1 to the value of i.
Endwhile
Print the values of Largest-so-far and Location.



Write down with Flowcharts

Flowcharts

- A flowchart is a graphical representation of a process or algorithm.
- It uses various symbols to denote different types of actions or steps in a process



- The **oval** represents the start and end points.
- The **diamond** represents the decision point
- The **rectangles** represent actions to be taken
- The **arrows** show the flow from one step to the next
- The **Parallelograms** show input into or output from the process

Types of Flowcharts

- **Process Flowchart**: Depicts the sequence of steps in a process or system.
- **Data Flow Diagram**: Shows the flow of data within a system, emphasizing data inputs and outputs.
- **System Flowchart**: Illustrates how data flows within a system and the system's various components.
- **Program Flowchart**: Used in programming to outline the logical flow of a program or algorithm.

Flowcharts are a useful tool

- Clarity: A flowchart algorithm can help to clarify complex processes
- Visualization: The graphical nature of flowcharts allows us to visualize the steps involved in a process
- Communication: Flowcharts are a universal language that can be understood by anyone
- Documentation: Flowcharts can be used as a form of documentation to help us remember the steps involved in a process or algorithm.
- Analysis: Flowcharts can help to analyze the effectiveness of a process or algorithm

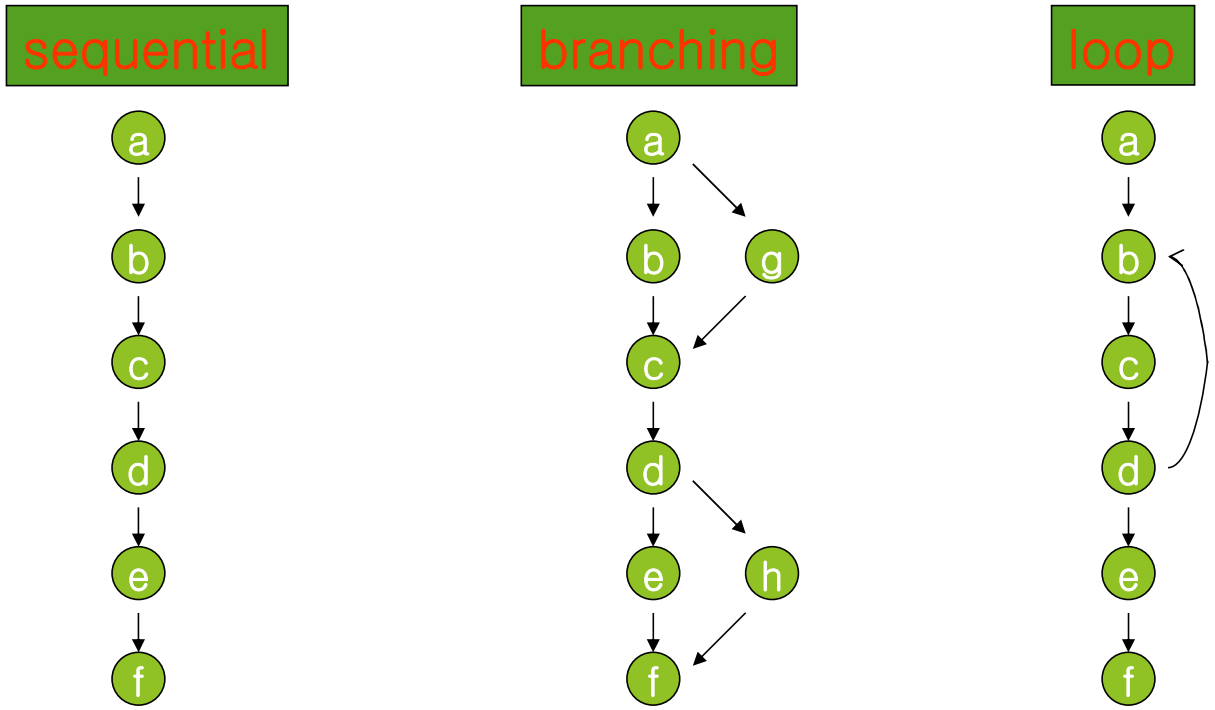
Disadvantage of Flowchart

- Limited Detail: Flowcharts provide a high-level overview of a process or algorithm
- Time-Consuming: Creating a flowchart can be time-consuming
- Complex Processes: Flowcharts may not be suitable for very complex processes
- Skill Required: Creating a flowchart requires a certain level of skill

Online tools for Flowchart

1. Lucidchart: a web-based tool for creating flowcharts, diagrams, and other visualizations.
2. Draw.io: a free web-based tool for creating flowcharts, diagrams, and other visualizations.
3. SmartDraw: It offers a user-friendly interface and a wide range of templates and symbols for creating different types of flowcharts.

Flow Control



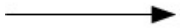
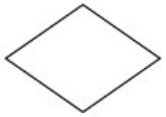
Algorithm Flowcharts (1)

- When tracing a flowchart, start at the start symbol and follow the flow lines to the end symbol
- Testing an algorithm at the flowchart stage is known as desk checking
- Flowcharts, pseudocode are program planning tools that are not dependent on the programming language being used

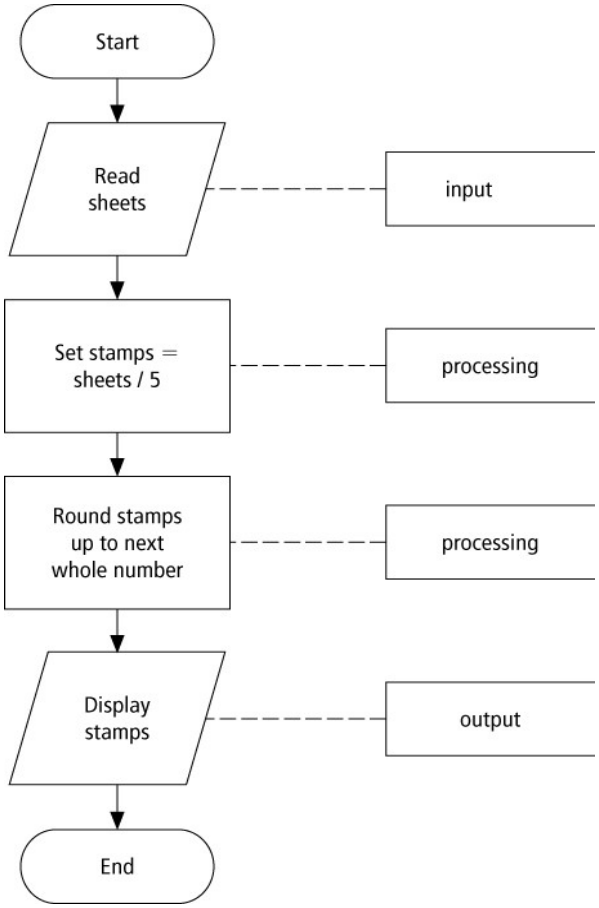
Algorithm Flowcharts (2)

- Flowcharts are time-consuming to write and difficult to update
  - For this reason, professional programmers are more likely to favor pseudocode
- Because flowcharts so clearly illustrate the logical flow of programming techniques,

Flowchart symbols

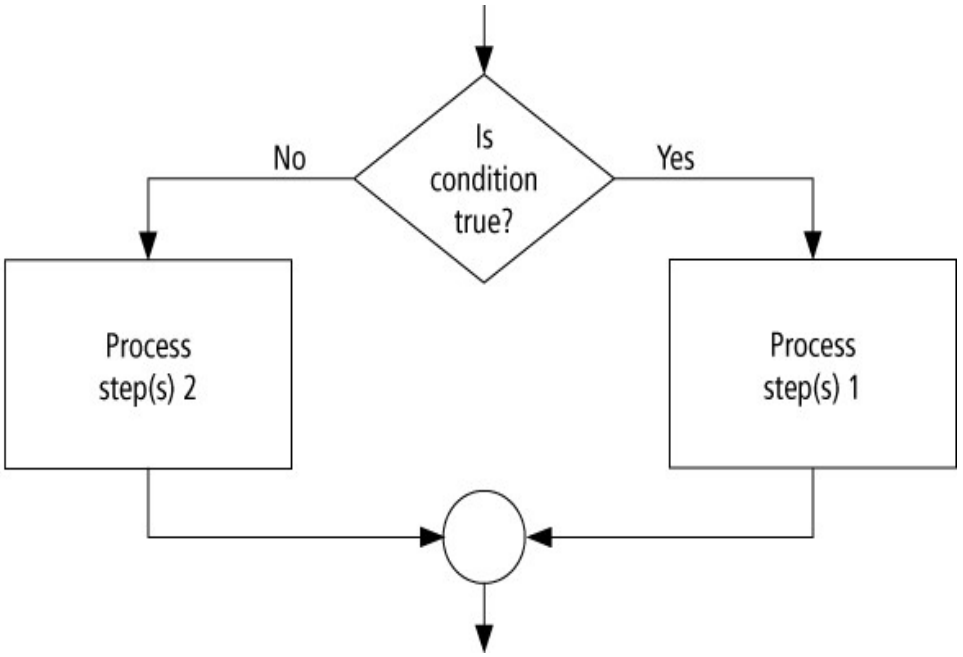
Symbol	Name	Meaning
	*Flowline*	Used to connect symbols and indicate the flow of logic.
	*Terminal*	Used to represent the beginning (Start) or the end (End) of a task.
	*Input/Output*	Used for input and output operations, such as reading and displaying. The data to be read or displayed are described inside.
	*Processing*	Used for arithmetic and data-manipulation operations. The instructions are listed inside the symbol.
	*Decision*	Used for any logic or comparison operations. Unlike the input/ouput and processing symbols, which have one entry and one exit flowline, the decision symbol has one entry and two exit paths. The path chosen depends on whether the answer to a question is "yes" or "no."

Flowchart example



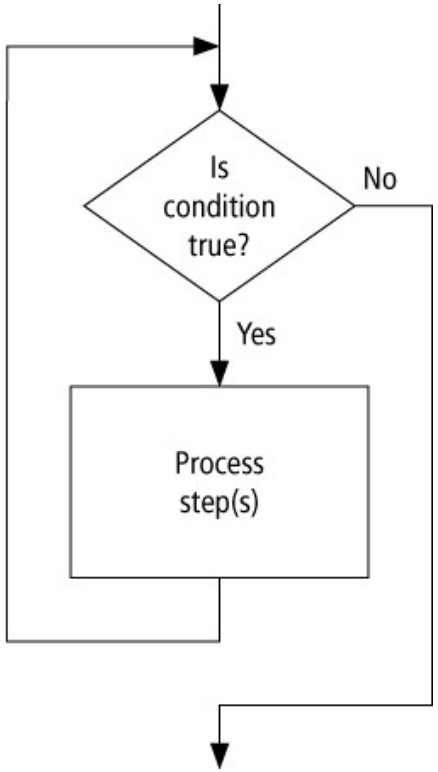
Decision flow chart

```
If condition is true Then
      Process step(s) 1
  Else
      Process step(s) 2
End If
```



Looping flow chart

```
Do While condition is true
    Process step(s)
Loop
```



Example

- ***Problem:*** Given a street number of a one-way street in New York City, decide the direction of the street, either eastbound or westbound
- ***Discussion:*** in New York City even numbered streets are Eastbound, odd numbered streets are Westbound

Pseudocode

Program: Determine the direction of a numbered NYC street

Get street

If street is even Then

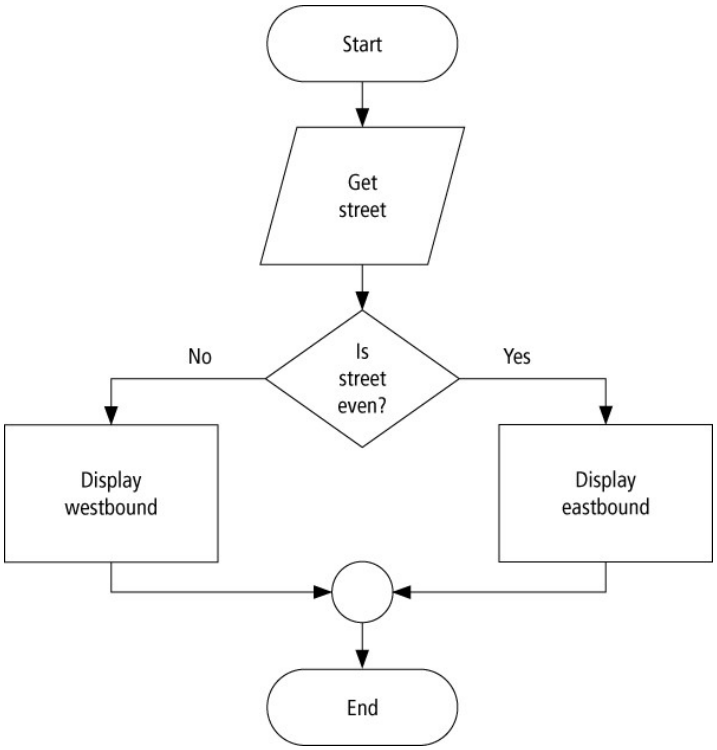
  Display Eastbound

Else

  Display Westbound

End If

Flowchart


Start
Get street
Is street even?
No
Yes
Display westbound
Display eastbound
End

Example

- ***Problem:*** Calculate and report the grade-point average for a class
- ***Discussion:*** The average grade equals the sum of all grades divided by the number of students

Input: Student grades

Processing: Find the sum of the grades; count the number of students; calculate average

Output: Average grade

Pseudocode

Program: Determine the average grade of a class

Initialize Counter and Sum to 0

Do While there are more data

    Get the next Grade

    Add the Grade to the Sum

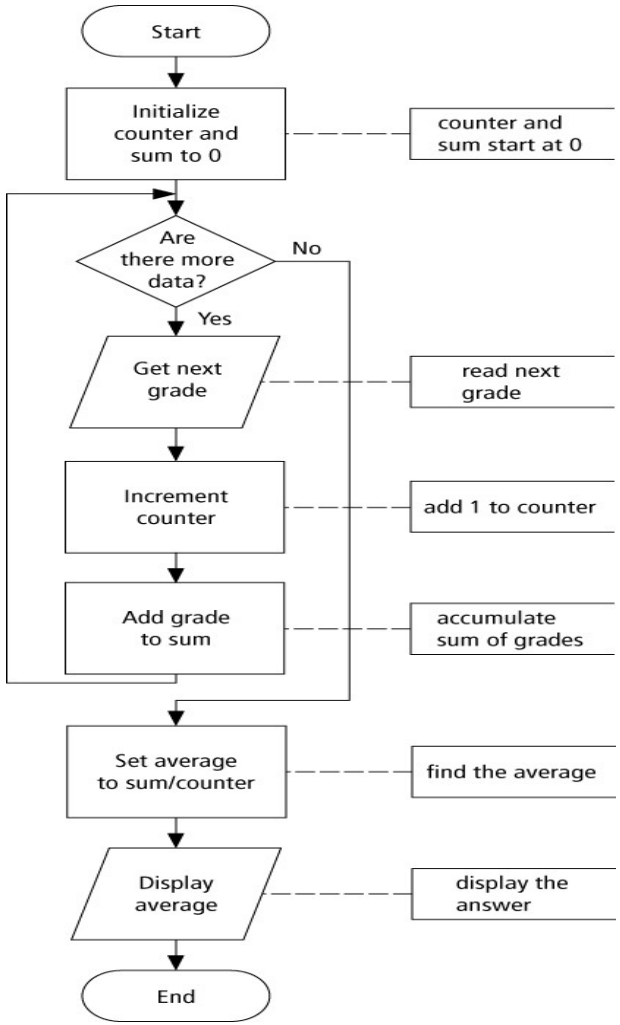
    Increment the Counter

Loop

Computer Average = Sum / Counter

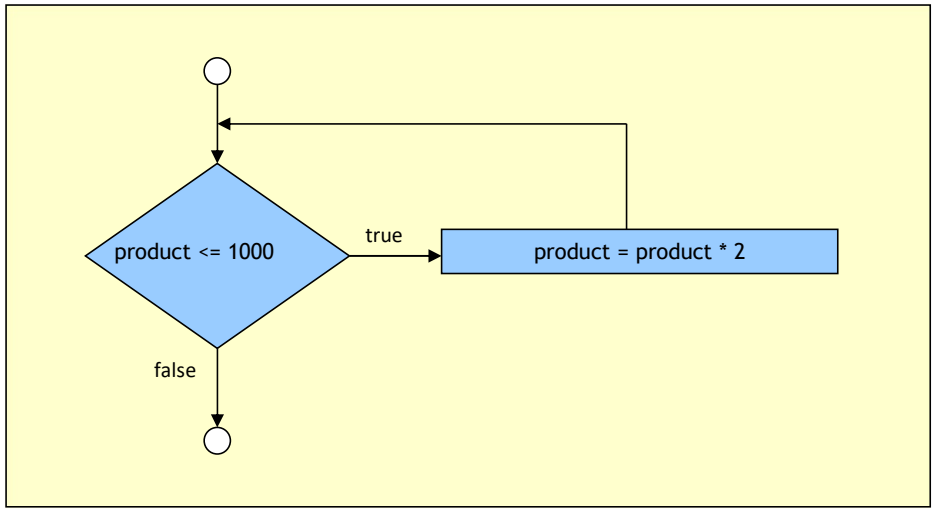
Display Average

Flowchart

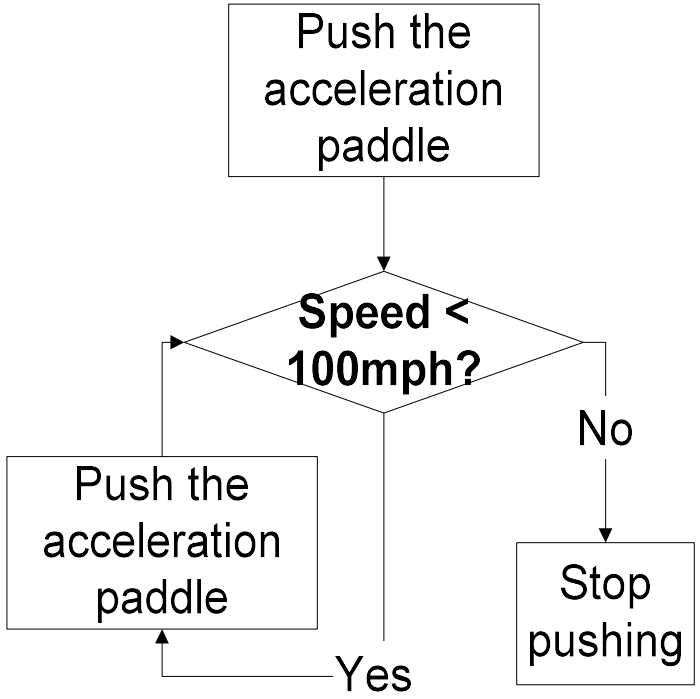

Start
Initialize counter and sum to 0
counter and sum start at 0
Are there more data?
No
Yes
Get next grade
read next grade
Increment counter
add 1 to counter
Add grade to sum
accumulate sum of grades
Set average to sum/counter
find the average
Display average
display the answer
End

While Repetition Structure

- Repetition structure
  - Allows the programmer to specify that an action should be repeated, depending on the value of a condition



while loop




```
Push the acceleration paddle;
while (speed is less than 100mph)
{
    push the acceleration paddle;
}
Release the paddle;
```

Exercise

Problem: Write a program that reads and displays the age of 10 people (one after another).

For this problem, we need a way to count how many people whose age have been processed (read and displayed). Therefore, we introduce a concept of *counter*, a variable used to count the number of people whose age have been processed by the program.

Exercise

```
Begin
   number of users giving his age = 1
   while (number of users giving his age <= 10)
    read the age from the user.
    print the user age.
    number of user giving his age + 1
   end_while
End
```

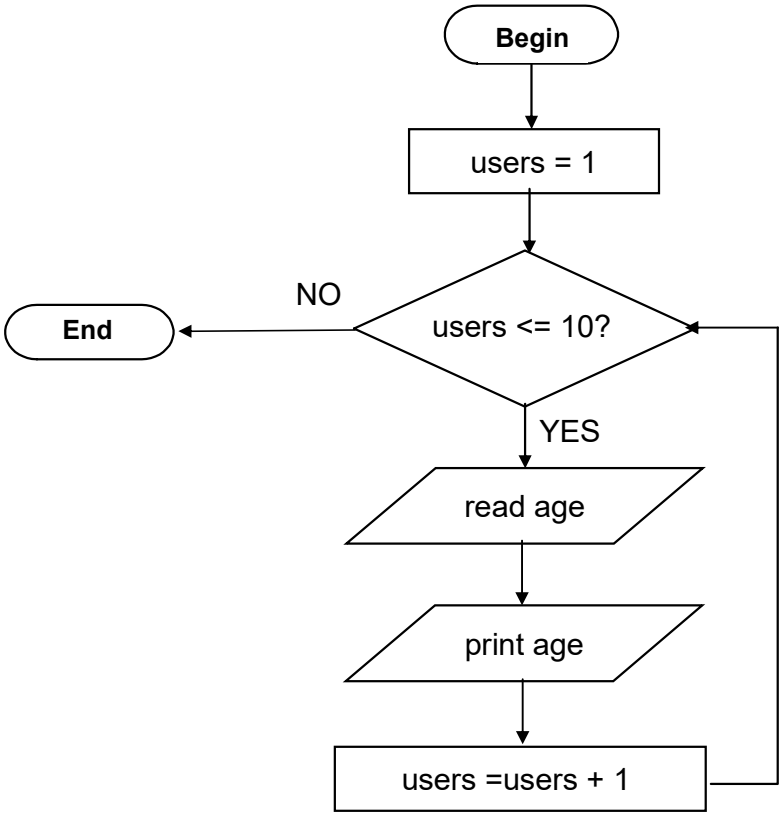
Counter initialisation

Loop condition

Updating counter

```
Begin
   users = 1
   while (users <= 10)
    read age
    print age.
    users = users + 1
   end_while
End
```

Exercise


Begin
users = 1
users <= 10?
NO
End
YES
read age
print age
users =users + 1

Subsequently..

```
Begin
   number of users giving his age = 0
   while (number of users giving his age < 10)
    read the age from the user.
    print the user age.
    number of user giving his age + 1
   end_while
End
```

You can start the counter with ZERO

The loop condition must less than the value it requires to stop

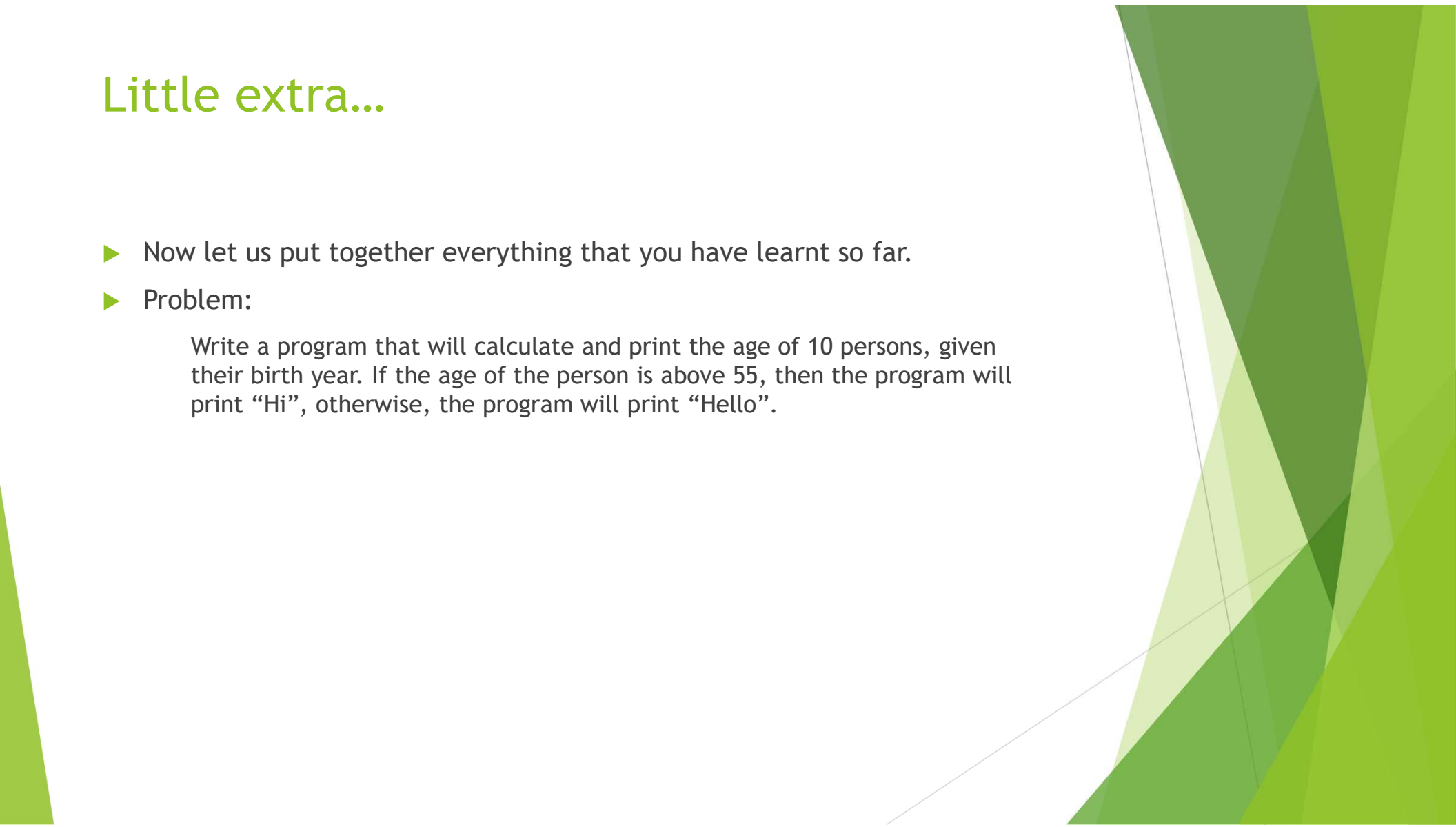
```
Begin
   users = 0
   while (users < 10)
    read age
    print age.
    users = users + 1
   end_while
End
```



Little extra…

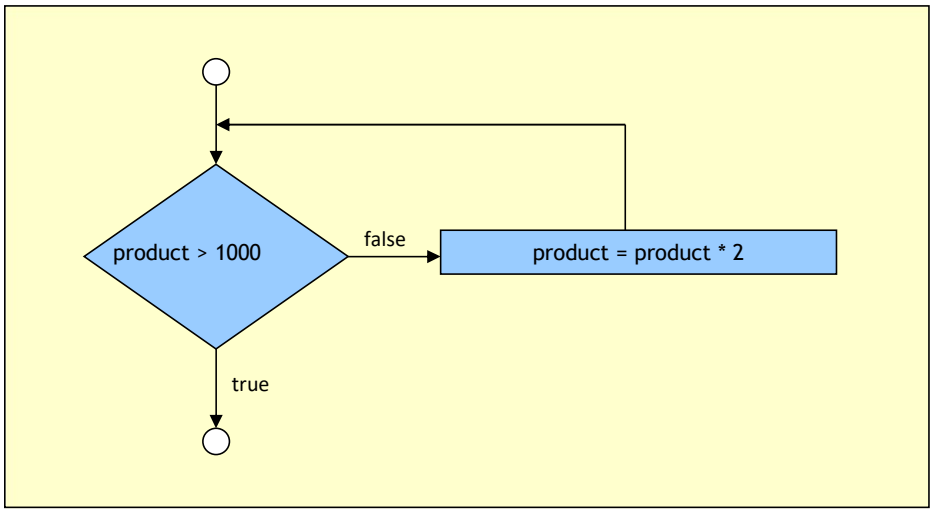
- Now let us put together everything that you have learnt so far.
- Problem:

  Write a program that will calculate and print the age of 10 persons, given their birth year. If the age of the person is above 55, then the program will print “Hi”, otherwise, the program will print “Hello”.



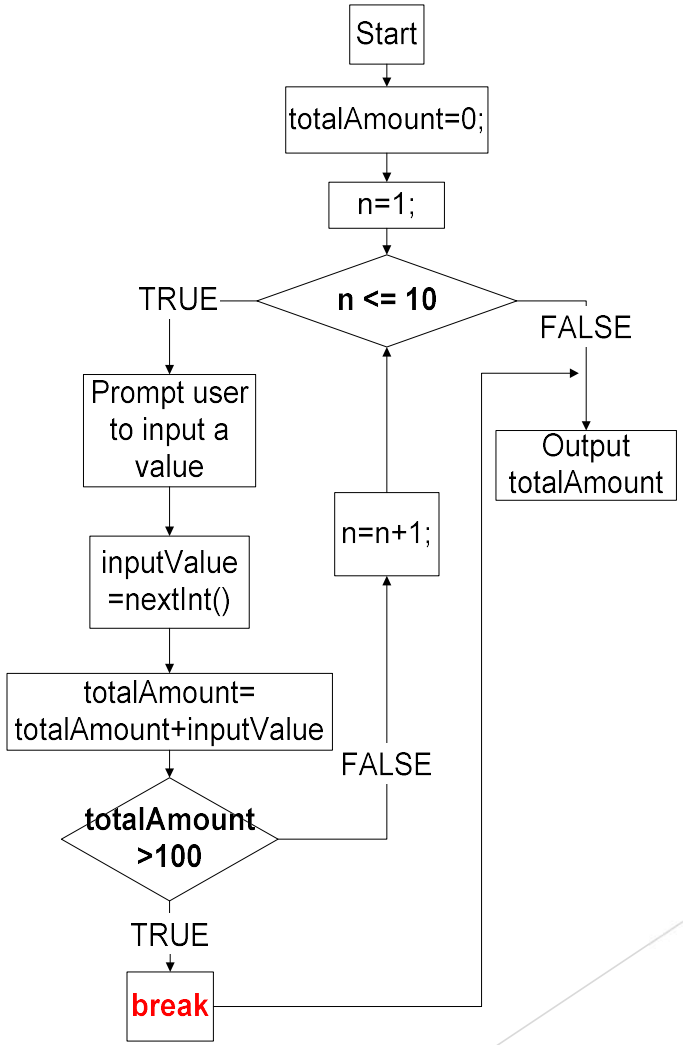
Do Until/for Loop Repetition Structure

- It tests a condition for falsity for repetition to continue.



Example

- Prompt the user for 10 input (integer values). Stop and output the total amount if the total amount is larger than 100.
- Input: 1,2,3,4,…,10
  - Complete 10 iterations
  - Output: 55
- Input: 50,40,30,20,…
  - Break the loop in iteration 3
  - Output: 120



Example

```
int inputValue, n, totalAmount=0;

for ( n=1; n<=10; n++)
{
    inputValue=keyboard.nextInt();
    totalAmount=totalAmount+inputValue;

    if ( totalAmount > 100 )
    {
        break;
    }
}

System.out.println(totalAmount);
```

